Family Outing Planner & Expense Ledger: Master Implementation Plan

Execution Roadmap & Gatekeeper Rules · Version 1.0

> **Gatekeeper Rule**
> This plan serves as the strict master roadmap for both you and your AI coding agents. **No step may begin until the previous step is fully completed, verified, and passes all unit/integration checks.**

Phase 0: Project Initialization & Workspace Setup

Initialize repository structure, ignore rules, and tooling environment.

- `[ ] Step 0.1` Initialize git repository with root structure (`/backend`, `/mobile`, `/skills`, `.clinerules`, `.aiignore`).
- `[ ] Step 0.2` Verify VS Code extensions (Java, Flutter, Dart, Lombok) and LSP diagnostics.
- `[ ] Step 0.3` Set up Neon PostgreSQL project instance and secure connection strings.

Phase 1: Backend Foundation (Spring Boot Modular Monolith)

Establish backend architecture, database connection, security, and Auth/User modules.

- `[ ] Step 1.1` Initialize Spring Boot 3.x project (Java 21, Gradle Kotlin DSL) with dependencies (Web, Data JPA, PostgreSQL, WebSocket, Security, JJWT).
- `[ ] Step 1.2` Configure database connection properties targeting Neon PostgreSQL and set up Hibernate DDL auto for development.
- `[ ] Step 1.3` Implement **Auth & User Module** enforcing the strict **1-to-1 User-to-Family binding** (JWT claims containing `family_id`).
- `[ ] Step 1.4` Write unit tests for user registration, family invitation token generation, and tenant isolation guards.

Phase 2: Core Domain Modules (Spring Boot)

Implement Calendar, Consensus (Voting), and Ledger modules adhering to domain-driven boundaries.

- `[ ] Step 2.1` Implement **Calendar Module**: Internal `UserAvailability` slots and overlap finder algorithm.
- `[ ] Step 2.2` Implement **Consensus Module**: Idea pitching, status state machine (`DRAFT`, `OPEN`, `FINALIZED`), and ranked-choice voting logic.
- `[ ] Step 2.3` Implement **Ledger Module**: `BigDecimal` expense tracking, transaction splits, and the deterministic **Min-Cash-Flow debt simplification algorithm**.
- `[ ] Step 2.4` Run comprehensive unit and integration tests across all modules (verifying ACID transactions).

Phase 3: Real-Time & Event Bus Layer

Configure internal Spring events, WebSockets, and internal notifications.

- `[ ] Step 3.1` Implement Spring `ApplicationEvent` publishers for domain transitions (e.g., `OutingFinalizedEvent`).
- `[ ] Step 3.2` Configure Spring WebSocket STOMP broker for live voting tally broadcasts.
- `[ ] Step 3.3` Implement internal notification listener writing unread records to the database without external push services.

Phase 4: Mobile App Foundation (Flutter)

Set up Flutter project, local Hive database, Dio client, and WebSocket connection.

- `[ ] Step 4.1` Initialize Flutter project with folder structure (`/lib/models`, `/lib/services`, `/lib/screens`, `/lib/providers`).
- `[ ] Step 4.2` Configure **Hive** local storage for offline caching of outings, calendars, and ledger states.
- `[ ] Step 4.3` Configure **Dio** client with interceptors for automatic JWT injection and offline optimistic mutation queuing.
- `[ ] Step 4.4` Configure `stomp_dart_client` for real-time WebSocket connection to the backend.

Phase 5: Mobile UI Implementation

Build the core user screens mapped to the architecture and offline synchronization state.

- `[ ] Step 5.1` Build **Dashboard / Home Screen** with calendar overview and quick navigation.
- `[ ] Step 5.2` Build **Availability Setup & Overlap View** screens.
- `[ ] Step 5.3` Build **Idea Board & Ranked-Choice Voting** screens.
- `[ ] Step 5.4` Build **Outing Ledger & Debt Settlement Settle-Up** screens.

Phase 6: End-to-End Testing & Deployment

Deploy backend to Vercel, connect Flutter client, and execute smoke tests.

- `[ ] Step 6.1` Create `Dockerfile` for Spring Boot fat JAR packaging compatible with Vercel containers.
- `[ ] Step 6.2` Deploy backend to Vercel and connect production Neon PostgreSQL database.
- `[ ] Step 6.3` Build release APK/Runner for Flutter and verify end-to-end sync, offline caching, and debt calculation.

> **How to use this plan with your agents**
> When prompting your coding agent in VS Code, reference the exact step to maintain strict focus and context:
>
> ```
> "We are currently on Phase 1, Step 1.3: Implement Auth & User Module enforcing strict 1-to-1 User-to-Family binding. Do not proceed to Step 1.4 until complete and tested."
> ```